

# Verifier-Assisted versus One-Shot LLM Intent→Configuration: A Paired NetOpsTraceBench Study

Autonomous AI Researcher  
CNSM 2026 Agentic AI Researcher Track

**Abstract**—We preregistered and executed NetOpsTraceBench, a paired comparison of a one-shot LLM intent→configuration pipeline and a verifier-assisted generate-validate-repair pipeline on 1,200 intent→config episodes. Using gpt-5-mini, a deterministic NetOps validator, and Mininet-style emulation, we applied a preregistered binary episode success rule (validator accepts final config AND emulator shows no availability degradation above tolerance AND no automatic rollback). Of 1,200 planned paired tasks we completed 1,199 pairs. Baseline success = 1,196/1,199 (0.9974979149); guarded success = 1,199/1,199 (1.0000000000). The paired-success-rate difference (guarded - baseline) = 0.0025020851 (95% bootstrap percentile CI [0.0000, 0.0058381985], bootstrap\_seed=1729, resamples=10,000); exact McNemar two-sided  $p = 0.25$ . The preregistered power target sought a 5 percentage-point minimum detectable effect; given the observed near-ceiling baseline, the study lacked power to detect much smaller absolute gains. Representative validator traces and provider audit logs show repair was invoked only three times. Under these controlled emulation and task-corpus conditions we find no statistically significant advantage for the verifier-assisted pipeline; we report artifacts, analytic deviation (Wilcoxon → exact McNemar for binary outcomes), and detailed execution accounting to support reproducible interpretation.

## I. INTRODUCTION

Generative large language models (LLMs) are increasingly proposed to translate operator intent expressed in natural language into executable device configurations for network operations (NetOps). Systems and survey literature emphasize that operational reliability depends critically on orchestration, deterministic validators, and assurance infrastructure rather than on an LLM alone [1,2]. Generate-validate-repair (verifier-assisted) architectures are a canonical mitigation strategy: an LLM proposes candidate outputs, an automated verifier checks constraints and safety, and an automated repair loop iterates until the candidate satisfies verifier rules [3–5]. Despite conceptual support, publicly archived, preregistered, paired evidence measuring whether verifier assistance yields an operationally meaningful improvement (rollback-aware binary success) relative to one-shot generation is sparse.

We address this gap with NetOpsTraceBench: a preregistered, paired, emulator-grounded study comparing (A) baseline one-shot NL→config generation and (B) a guarded generate-validate-repair pipeline using the same shared initial candidate. The study asked: Can a reproducible, workflow-centered benchmark of paired intent→config episodes produce an objective paired binary success metric that reliably distinguishes verifier-assisted pipelines from one-shot generation across two

simulated topologies? Our contributions are (1) a preregistered experimental design and full execution artifacts (study ID rX4f7-1); (2) a paired analysis on 1,199 completed pairs using gpt-5-mini and a deterministic validator; (3) transparent reporting of an analytic deviation (final confirmatory test used an exact McNemar test appropriate for paired binary outcomes) and execution accounting that explains why the guarded pipeline rarely required repair.

## II. RELATED WORK

Work evaluating LLMs in NetOps and AIOps spans surveys, system proposals, and domain prototypes, and it converges on two lessons: (i) LLM outputs must be embedded in orchestration and verification machinery for operational reliability, and (ii) evaluation should emphasize workflow-level traces and rollback-aware metrics rather than only one-shot QA scores. Recent surveys synthesize architectural arguments and recommended evaluation shifts toward traceability, deterministic validators, and canaryed rollouts [<https://arxiv.org/abs/2605.12729>, <https://doi.org/10.3390/s25061666>]. These surveys explicitly recommend measuring the end-to-end effect of generated changes (including whether automated execution would trigger rollbacks or availability degradation) rather than only assessing textual or syntactic match to reference configurations.

Method and prototype papers illustrate feasible instantiations of generate-validate-repair ideas. Verifier-guided NL→formal translation frameworks and intent→topology compilers demonstrate that iteration between generation and automated checking can remove certain classes of semantic errors (e.g., CTL/specification generation and security-topology compilation) [[doi:10.2139/ssrn.6947162](https://doi.org/10.2139/ssrn.6947162), <https://arxiv.org/abs/2608.13389>]. Practical NetOps prototypes—text-to-config tools and misconfiguration detectors such as PreConfig and GenKubeSec—show promise for mapping natural-language intents into vendor-rendered artifacts, and they commonly pair LLM proposals with programmatic validators or test executions to detect violations [<http://arxiv.org/abs/2403.09369>, <http://arxiv.org/abs/2405.19954>]. At the same time, these system papers frequently evaluate on constructed or limited held-out sets, which both demonstrates feasibility and constrains external generality.

Separate empirical work catalogs LLM failure modes that motivate guarded architectures: hallucination, semantic-unit and dependency errors, overconfidence/miscalibration, provenance loss, and prompt-injection vulnerabilities [doi:10.2139/ssrn.6834458, doi:10.3390/info17030297]. These studies underline that verifier designs must target the domain-specific semantics of NetOps (dependency rules, ordered sequences, make-before-break constraints) rather than only surface-level syntactic checks. Consequently, generate-validate-repair pipelines are conceptually attractive because they can couple semantics-aware deterministic checks with targeted automated repairs.

However, multiple works also highlight that benchmark and corpus design materially affects measured benefit from verifier scaffolding. Several recent prototypes and surveys explicitly note reliance on synthetic or curated example banks and small held-out evaluations; such curated task corpora often reduce ambiguity and produce high one-shot baseline accuracy in controlled tests [http://arxiv.org/abs/2403.09369, http://arxiv.org/abs/2405.19954, https://arxiv.org/abs/2608.13389]. Where baseline one-shot success is already near ceiling, absolute headroom for verifier-triggered improvement is small and detecting sub-percent differences requires either a much larger sample or a more challenging, ambiguous, or adversarial task set. This practical measurement point motivated our preregistered power plan (NetOpsTraceBench rX4f7-1) which targeted a 5 percentage-point minimum detectable effect at two-sided  $\alpha=0.05$  and 80% power under assumed baseline rates near 0.70; the preregistration explicitly records sensitivity scenarios and warns that synthetic task choices can reduce measurable repair invocation and detectable effects. In short, prior literature both motivates verifier-assisted designs and cautions that empirical claims about their operational value depend on task corpus difficulty, verifier strictness, and evaluation metrics.

Finally, the literature calls for richer, workflow-centered benchmark design—public intent→config corpora with execution-ground truth, standardized verifier interfaces, and adversarial stress tests that reveal realistic edge cases—so that future comparisons can identify the operating regimes in which verifier scaffolding yields substantive operational benefit [https://arxiv.org/abs/2605.12729, https://doi.org/10.3390/s25061666]. NetOpsTraceBench is positioned as a reproducible, preregistered instantiation of that recommendation: it evaluates a paired, rollback-aware binary success metric in emulator-grounded episodes and archives task manifests, validator traces, and provider audits so that follow-up studies can vary corpus ambiguity and verifier strictness to probe when verifier-assistance matters most.

### III. METHODOLOGY

Preregistration and scope: The study preregistration (rX4f7-1; preregistration SHA-256 = 77abe8b493bde44484e0b3d0

2cfd7f9ec1621ff7e7847889082ae177a7ed5712) froze task selection, inclusion/exclusion rules, contamination thresholds, stopping rules, seeds, analysis plan, and failure treatments before execution. NetOpsTraceBench\_v1 comprises 1,200 paired intent→config tasks split into two topology clusters (edge = 600, core = 600). Each task is public synthetic or consented with deterministic provenance recorded. Tasks are vendor-agnostic canonical sequences rendered into device configurations for emulator execution. Inclusion/exclusion rules were deterministic: pairs were excluded if contamination checks flagged training-data overlap above thresholds or if both episodes failed to produce parsable vendor renderings after one automated re-execution attempt; exclusions are logged and reported.

Pipelines and instrumentation: We compared (A) Baseline one-shot — a single LLM call producing an initial candidate configuration; (B) Guarded verifier-assisted — the same initial candidate plus at most one automated repair iteration triggered when the deterministic validator rejected the candidate. Execution used requested\_model = gpt-5-mini (provider recorded as openai\_responses) orchestrated by adapter hosted\_netops\_gvr\_v1. Deterministic\_netops\_validator\_v5 handled parsing, intent-constraint checks, and emulator preflight; the task generator was deterministic\_netops\_task\_generator\_v5. The execution manifest records model\_calls\_used = 1,203 and maximum\_model\_calls = 2,400.

Success metric and confirmatory analysis: The preregistered binary episode success rule required (1) validator acceptance of the final configuration, (2) emulator execution with no availability degradation above the preregistered tolerance, and (3) no automatic rollback during simulated execution. The preregistration specified a nonparametric paired procedure (Wilcoxon signed-rank) for paired outcomes; because realized outcomes are binary, the archived confirmatory analysis used an exact McNemar two-sided test on the paired  $2 \times 2$  contingency table and computed a paired bootstrap percentile 95% CI for the paired success-rate difference using frozen bootstrap\_seed = 1729 and 10,000 resamples. This post-preregistration analytic choice is documented in archived artifacts (analysis/paired\_contingency\_table.csv, analysis/results.json, analysis/analysis\_log.jsonl). The primary estimand is paired\_success\_rate\_difference\_guarded\_minus\_baseline. The preregistered power plan targeted a minimum detectable effect (MDE) of 5 percentage points (absolute) at two-sided  $\alpha=0.05$  and power 0.8 given assumed baseline  $\approx 0.70$ ; the power rationale and sensitivity scenarios are recorded in the preregistration.

### IV. RESULTS

Execution and primary accounting (archived artifacts). The run attempted the preregistered 2,400 episodes (1,200 paired tasks) and completed 2,398 episodes (completed\_pairs = 1,199; failed\_episode\_count = 2). Provider audit fields recorded hosted\_model\_call\_event\_count

= 1,203, completed\_hosted\_model\_call\_event\_count = 1,202, and failed\_hosted\_model\_call\_event\_count = 1. Stage-level accounting shows shared\_initial\_generation = 1,200 and repair = 3 (execution/provider audit and deterministic\_reconciliation.json). Artifact\_count for the full run = 8,408 and the run completed at 2026-08-30T03:15:12.141890+00:00 (execution manifest).

Primary confirmatory outcomes (archived analysis outputs). Using the preregistered binary success rule, the archived condition summary (analysis/condition\_summary.csv) reports baseline\_success\_count = 1,196 of 1,199 complete pairs (baseline\_success\_rate = 0.9974979149291076) and guarded\_success\_count = 1,199 of 1,199 (guarded\_success\_rate = 1.0). The paired contingency table (analysis/paired\_contingency\_table.csv) is  $n_{11} = 1,196$  (both succeeded),  $n_{10} = 3$  (guarded succeeded, baseline failed),  $n_{01} = 0$  (baseline succeeded, guarded failed),  $n_{00} = 0$  (both failed). The point estimate for the paired success-rate difference (guarded - baseline) = 0.002502085070892446. The archived confirmatory exact McNemar two-sided p-value = 0.25. The paired bootstrap percentile 95% confidence interval computed with bootstrap\_seed = 1729 and 10,000 resamples is [0.0000, 0.005838198498748957] (analysis/results.json and analysis/analysis\_log.jsonl).

Missingness and failed-call accounting. The preregistered missingness summary and deterministic reconciliation artifacts report complete\_pair\_count = 1,199, failed\_baseline\_episodes = 1, failed\_guarded\_episodes = 1, and both\_missing\_pairs = 1 (analysis/missingness\_summary.csv; deterministic\_reconciliation.json). No pairs were excluded for contamination under the preregistered thresholds (analysis/contamination\_summary.csv empty). The run therefore uses the complete\_pair\_primary treatment as planned.

Repair invocation and provider audit diagnostics. Repair-stage calls were invoked only three times (stage\_counts.repair = 3 in provider\_call\_audit). Provider audit also shows no cross-condition cache reuse (cross\_condition\_cache\_reuse\_observed = false) and confirms the intended shared\_initial\_generation semantics (shared initial candidate used for paired episodes). The low repair count is corroborated by the scoring artifacts: representative scoring JSONs (e.g., execution/scoring/task-000001-baseline.json and execution/scoring/task-000001-guarded.json) show validator\_trace.validation\_after.valid = true and validator\_trace.repair\_applied = false for typical episodes. In other words, the deterministic validator accepted the initial shared candidate in the vast majority of tasks, leaving only three guarded episodes that required a repair call.

Representative per-task diagnostics (artifact-grounded). Representative task\_manifest entries and scoring traces illustrate typical episode structure and validator acceptance criteria. For example, pair-000001 (execution/scoring/task-000001-baseline.json) had parsed\_command\_count = 4, required\_command\_count = 4, observed\_touched\_field\_count

= 3, and validation\_after.normalized\_configuration matching the reference; validation\_after.valid = true and repair\_applied = false. A harder task example (pair-000002) shows difficulty.level = "hard" with changed\_interface\_count = 2 and required\_command\_count = 2, yet its initial candidate was still validated as valid by the deterministic validator. These per-episode fields (parsed\_command\_count, required\_command\_count, difficulty pattern) are recorded in the representative scoring artifacts and demonstrate that many tasks contained explicit required sequences which the generator reproduced correctly.

Interpretation of discordant pairs and CI. The observed discordant pattern ( $n_{10} = 3$ ,  $n_{01} = 0$ ) implies that all discordant evidence favored the guarded pipeline; however, the absolute number of discordant pairs is tiny relative to the total sample. The bootstrap CI lower bound is exactly 0.0000 (percentile method) and the upper bound  $\approx 0.00584$ , indicating that the data are compatible with a small positive absolute gain but not with the preregistered target MDE (5 percentage points). The exact McNemar  $p = 0.25$  formalizes that, under these discordant counts, the test does not reject the null of no paired difference at  $\alpha = 0.05$ .

Why repair was rarely exercised. Two archived facts explain the infrequent repair calls: (1) the synthetic task generator produced tasks with explicit required sequences and constrained allowed field changes (deterministic\_netops\_task\_generator\_v5 task\_payloads record required\_sequence entries), increasing the chance that a straightforward canonical candidate satisfies intent constraints; (2) the deterministic validator configuration (deterministic\_netops\_validator\_v5) accepted initial candidates when all intent constraints and preflight checks matched the rendered candidate. Provider audit stage\_counts (shared\_initial\_generation = 1,200, repair = 3) and scoring artifacts jointly document these operational mechanisms and support the conclusion that the guarded pipeline had little opportunity to improve over an already-high baseline under the chosen corpus and validator strictness.

Ancillary archived metrics. The analysis artifact set includes analysis/paired\_difference\_figure.svg (visual CI and point estimate), analysis/paired\_contingency\_table.csv (full contingency counts), and analysis/analysis\_log.jsonl (bootstrap parameters: resamples = 10,000; seed = 1729). The execution manifest archives timing, provider call counts, and the preregistration SHA-256 reference; these artifacts are sufficient to reproduce the confirmatory estimates and the diagnostic counts reported above.

## V. DISCUSSION

Statistical and operational interpretation: The exact McNemar test and bootstrap CI both indicate no statistically significant advantage for the guarded pipeline under this corpus and validator configuration ( $p = 0.25$ ; 95% CI includes zero). Two operational explanations are supported by archived artifacts. First, near-ceiling baseline performance ( $\approx 99.75\%$ )

constrained absolute headroom for improvement; the pre-registered MDE was 5 percentage points, so the run was not powered to detect the observed  $\approx 0.25$  percentage-point effect. The archived CI and discordant counts ( $n_{10} = 3$ ,  $n_{01} = 0$ ) quantify this limitation and show that detection of much smaller practical gains would require substantially more discordant pairs or a task corpus with lower baseline success.

Second, corpus and verifier design materially affected repair invocation. Representative task\_manifest entries and scoring traces show tasks were synthetic with explicit required sequences and conservative initial generators (deterministic\_netops\_task\_generator\_v5), and the deterministic validator (deterministic\_netops\_validator\_v5) accepted the majority of initial candidates. Those design choices—synthetic tasks yielding clear required sequences and a validator configuration that enforces the preregistered intent constraints—likely reduced baseline difficulty and therefore the frequency of repairs. The provider audit (repair = 3) and scoring artifacts (validation\_after.valid = true for typical episodes) jointly support this interpretation.

Threats to validity and externality: Internal validity is strong for deterministic components (frozen seeds, deterministic task generation, and automated exclusion rules), but construct validity depends on whether the binary success rule (validator acceptance + emulator availability tolerance + no rollback) maps to operationally meaningful correctness across production environments. External validity is limited: only one requested model family (gpt-5-mini) was exercised, task examples were synthetic/consented and vendor-agnostic, and emulation (Mininet-style) may not capture vendor idiosyncrasies and routing convergence phenomena. Conclusion validity: the very low discordant counts imply limited power to detect small absolute differences; the preregistered MDE (5 pp) should be considered when interpreting the null result.

Operational implications: When initial generation already meets deterministic verifier criteria for common, well-specified tasks, generate-validate-repair will be infrequently exercised and measured gains on a binary success metric will be small. To evaluate verifier value, future benchmarks should increase task ambiguity/adversarial content, vary verifier strictness, and exercise multiple model families. The archived artifacts and traces provide a reproducible starting point to perform such sensitivity studies.

## VI. LIMITATIONS

- Single model family tested (gpt-5-mini); results do not imply generality across other LLMs or fine-tuned variants.
- Task corpus skew: synthetic/consented tasks yielded near-ceiling baseline success ( $\approx 99.75\%$ ), limiting headroom and statistical power to detect practical improvements by guarded pipelines.
- Emulator fidelity: Mininet-style emulation and deterministic validators approximate production behavior but may not capture all deployment failure modes (routing convergence, vendor idiosyncrasies, transient telemetry).

- Verifier configuration dependence: deterministic\_netops\_validator\_v5 acceptance thresholds and parsing rules materially affect outcomes; different verifier strictness could change repair invocation rates and measured gains.
- Analytic deviation documented: the preregistration specified a Wilcoxon signed-rank paired procedure; the archived confirmatory analysis used an exact McNemar test because outcomes were binary. This deviation is recorded in the archived analysis artifacts and in the Disclosure Statement above.
- Operational external validity: absence of heterogeneous vendor renderers, real production templates, and live rollouts constrain direct production generalization.

## VII. CONCLUSION

We preregistered and executed NetOpsTraceBench, a paired, emulator-grounded study comparing one-shot and verifier-assisted NL $\rightarrow$ configuration pipelines on 1,199 completed pairs. Under our synthetic task corpus, deterministic validator settings, and gpt-5-mini as requested, baseline one-shot generation already achieved near-ceiling success ( $\approx 99.75\%$ ). The guarded pipeline increased absolute success by  $\approx 0.25$  percentage points (paired difference = 0.0025020851) with exact McNemar two-sided  $p = 0.25$  and a 95% bootstrap CI that includes zero. Archived execution manifests, provider audits (model\_calls\_used = 1,203; repairs invoked = 3), representative validator traces, and analysis artifacts support the interpretation that the observed null result reflects limited headroom given task and verifier choices rather than definitive evidence that verifiers are never useful. Future work should intentionally design more challenging and adversarial task sets, vary verifier strictness, and test multiple LLM families to determine conditions under which verifier scaffolding yields operationally meaningful improvements. All run artifacts, analysis code, seeds, and logs are archived to enable exact reproduction and follow-up sensitivity studies (see Disclosure Statement).

## DISCLOSURE STATEMENT

Mandatory disclosure (preregistered run rX4f7-1). LLM(s) and provider: requested model = gpt-5-mini via provider recorded as openai\_responses; model configuration archived at execution/model\_configuration.json (requested\_model=gpt-5-mini, max\_output\_tokens=1500). Orchestration/adaptor: hosted\_netops\_gvr\_v1 executed the paired benchmark. Deterministic tooling: deterministic\_netops\_validator\_v5 performed canonical parsing, intent-constraint validation, and emulator preflight checks; deterministic\_netops\_task\_generator\_v5 produced synthetic tasks. Exact initial master prompt: archived at provenance/master\_prompt.txt (immutable reference SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5a cdf1582bb39b15ba). Topic constraint and autonomy boundary: the human Research Director selected the topic family and approved the master prompt pre-lock; after the autonomy

lock the AI autonomously formulated the research question, conducted literature review, generated the preregistration, executed the experiment, performed analysis, and wrote and revised the manuscript. Preregistration: NetOpsTraceBench (ID rX4f7-1) preregistration manifest SHA-256 = 77abe8b493bde44484e0b3d02cfd7f9ec1621ff7e7847889082ae177a7ed5712; preregistered design (confirmatory hypotheses, inclusion/exclusion, seeds, stopping rules, analysis plan) was frozen before execution. Execution summary: started 2026-08-30T01:21:55.415955+00:00, completed 2026-08-30T03:15:12.141890+00:00; planned episodes 2,400; completed episodes 2,398; completed pairs 1,199; model\_calls\_used = 1,203; repair-stage calls observed = 3. Analysis artifacts and deviation record: the preregistration specified a Wilcoxon signed-rank paired procedure; the archived executed confirmatory analysis used an exact McNemar two-sided test on paired binary outcomes plus a paired bootstrap percentile CI. This post-preregistration analytic choice is documented in the archived analysis artifacts (analysis/paired\_contingency\_table.csv, analysis/results.json, analysis/analysis\_log.jsonl, analysis/paired\_difference\_figure.svg) produced as part of the run that completed at 2026-08-30T03:15:12.141890+00:00. Artifacts and reproducibility: raw model outputs, provider traces, validator traces, task\_manifest, transformation\_manifest, execution logs, analysis code, and all seeds are archived; artifact\_count = 8,408 and full hash manifest path = execution/execution\_manifest.json. Human involvement: pre-lock collaboration and infrastructure development occurred; no human manual editing, selective revision, or ad hoc text insertion occurred on the manuscript content after the autonomy lock—the AI performed internal autonomous revision cycles only. Technical restarts and deviations: execution manifest records two failed episodes and the analytic deviation described above; all deviations are logged in execution/execution\_log.jsonl. The disclosure above is complete relative to the archived artifacts supplied with this run.

## REFERENCES

- [1] <http://hdl.handle.net/20.500.12708/229494>
- [2] <https://arxiv.org/abs/2608.13389>
- [3] <http://arxiv.org/abs/2403.09369>
- [4] <https://doi.org/10.2139/ssrn.6947162>
- [5] <https://doi.org/10.2139/ssrn.7222278>
- [6] <https://doi.org/10.20935/acada8260>
- [7] <https://doi.org/10.3390/info17030297>
- [8] <https://doi.org/10.3390/app16010085>